Implementing Lambdas with Environments: Closures

CS 350

Dr. Joseph Eremondi

March 24, 2025

Outline

Broad Goals

The Details

But Professor, When Will I Ever Use This?

Broad Goals

Overview

Goals

Overview

Goals

- Implement an interpreter for a Curly variant with functions

Overview

Goals

- Implement an interpreter for a Curly variant with functions
  - Add a Lambda feature

Overview

Goals

- Implement an interpreter for a Curly variant with functions
    - Add a Lambda feature
    - Allow functions to be taken or returned by functions

Overview

Goals

- Implement an interpreter for a Curly variant with functions
  - Add a Lambda feature
  - Allow functions to be taken or returned by functions
  - Allow function calls on arbitrary expressions, not just symbols

Overview

Goals

- Implement an interpreter for a Curly variant with functions
  - Add a Lambda feature
  - Allow functions to be taken or returned by functions
  - Allow function calls on arbitrary expressions, not just symbols
- Use Environments to make the implementation efficient

Overview

Goals

- Implement an interpreter for a Curly variant with functions
  - Add a Lambda feature
  - Allow functions to be taken or returned by functions
  - Allow function calls on arbitrary expressions, not just symbols
- Use Environments to make the implementation efficient
- Replicate the behaviour of the substitution-based interpreter

Overview

Goals

- Implement an interpreter for a Curly variant with functions
  - Add a Lambda feature
  - Allow functions to be taken or returned by functions
  - Allow function calls on arbitrary expressions, not just symbols
- Use Environments to make the implementation efficient
- Replicate the behaviour of the substitution-based interpreter

Key Concepts

Overview

Goals

- Implement an interpreter for a Curly variant with functions
  - Add a Lambda feature
  - Allow functions to be taken or returned by functions
  - Allow function calls on arbitrary expressions, not just symbols
- Use Environments to make the implementation efficient
- Replicate the behaviour of the substitution-based interpreter

Key Concepts

- Definition of a closure

Overview

Goals

- Implement an interpreter for a Curly variant with functions
  - Add a Lambda feature
  - Allow functions to be taken or returned by functions
  - Allow function calls on arbitrary expressions, not just symbols
- Use Environments to make the implementation efficient
- Replicate the behaviour of the substitution-based interpreter

Key Concepts

- Definition of a closure
- Static and Dynamic Scope for first-class functions

The Details

The Problem: Practical

- Each substitution is $\mathcal{O}(n)$ where n is the number of nodes in the function body AST

The Problem: Practical

- Each substitution is $\mathcal{O}(n)$ where n is the number of nodes in the function body AST
- This is *in addition* to the cost of actually evaluating the function

The Problem: Practical

- Each substitution is $\mathcal{O}(n)$ where n is the number of nodes in the function body AST
- This is *in addition* to the cost of actually evaluating the function
  - Very slow!

The Problem: Practical

- Each substitution is $\mathcal{O}(n)$ where n is the number of nodes in the function body AST
- This is *in addition* to the cost of actually evaluating the function
  - Very slow!
- Want a way to have $\mathcal{O}(1)$ function calls

The Problem: Practical

- Each substitution is $\mathcal{O}(n)$ where n is the number of nodes in the function body AST
- This is *in addition* to the cost of actually evaluating the function
  - Very slow!
- Want a way to have $\mathcal{O}(1)$ function calls
  - Not including the time to evaluate the function body

The Problem: Practical

- Each substitution is $\mathcal{O}(n)$ where n is the number of nodes in the function body AST
- This is *in addition* to the cost of actually evaluating the function
  - Very slow!
- Want a way to have $\mathcal{O}(1)$ function calls
  - Not including the time to evaluate the function body
- Substitution is forgetful

The Problem: Practical

- Each substitution is $\mathcal{O}(n)$ where n is the number of nodes in the function body AST
- This is *in addition* to the cost of actually evaluating the function
  - Very slow!
- Want a way to have $\mathcal{O}(1)$ function calls
  - Not including the time to evaluate the function body
- Substitution is forgetful
  - Just replaces function variable with expression

The Problem: Practical

- Each substitution is $\mathcal{O}(n)$ where n is the number of nodes in the function body AST
- This is *in addition* to the cost of actually evaluating the function
  - Very slow!
- Want a way to have $\mathcal{O}(1)$ function calls
  - Not including the time to evaluate the function body
- Substitution is forgetful
  - Just replaces function variable with expression
  - Not very useful for debugging

The Problem: Theoretical

- Interpreter serves as a *definition* for a language

The Problem: Theoretical

- Interpreter serves as a *definition* for a language
- Want to use interpreters to say what a program *means*

The Problem: Theoretical

- Interpreter serves as a *definition* for a language
- Want to use interpreters to say what a program *means*
- What does `{+ x y}` mean?

The Problem: Theoretical

- Interpreter serves as a *definition* for a language
- Want to use interpreters to say what a program *means*
- What does `{+ x y}` mean?
    - Substitution-interpreter: error

The Problem: Theoretical

- Interpreter serves as a *definition* for a language
- Want to use interpreters to say what a program *means*
- What does `{+ x y}` mean?
  - Substitution-interpreter: error
  - But this program isn't meaningless!

The Problem: Theoretical

- Interpreter serves as a *definition* for a language
- Want to use interpreters to say what a program *means*
- What does `{+ x y}` mean?
  - Substitution-interpreter: error
  - But this program isn't meaningless!
    - Meaning depends on *context*

The Solution: Environments

- Data structure for *deferred substitution*

The Solution: Environments

- Data structure for *deferred substitution*
    - List of variable/value pairs

The Solution: Environments

- Data structure for *deferred substitution*
    - List of variable/value pairs
- Intuition:

The Solution: Environments

- Data structure for *deferred substitution*
    - List of variable/value pairs
- Intuition:
    - Instead of replacing all `x` with value `v`, keep a list of replacements you need to do

The Solution: Environments

- Data structure for *deferred substitution*
  - List of variable/value pairs
- Intuition:
  - Instead of replacing all `x` with value `v`, keep a list of replacements you need to do
  - When you interpret `x`, check the environment before raising an error

The Solution: Environments

- Data structure for *deferred substitution*
  - List of variable/value pairs
- Intuition:
  - Instead of replacing all `x` with value `v`, keep a list of replacements you need to do
  - When you interpret `x`, check the environment before raising an error
  - If there's an entry for `x` in the environment, return that

The Solution: Environments

- Data structure for *deferred substitution*
    - List of variable/value pairs
- Intuition:
    - Instead of replacing all `x` with value `v`, keep a list of replacements you need to do
    - When you interpret `x`, check the environment before raising an error
    - If there's an entry for `x` in the environment, return that
        - Error otherwise

The Solution: Environments

- Data structure for *deferred substitution*
  - List of variable/value pairs
- Intuition:
  - Instead of replacing all x with value v, keep a list of replacements you need to do
  - When you interpret x, check the environment before raising an error
  - If there's an entry for x in the environment, return that
    - Error otherwise
    - Means reference to undefined variable

The Environment Data Structure

- Data structure associating variable keys with Values

The Environment Data Structure

- Data structure associating variable keys with Values

The Environment Data Structure

- Data structure associating variable keys with Values

```
(define-type-alias Binding (Symbol * Value))
(define-type-alias Env (Listof Binding))

;; Environment is either empty or extended env
(define mt-env : Env
  empty)
(define (extend [bnd : Binding]
                [env : Env])
        : Env
  (cons bnd env))
```

- Textbook uses hashtables

The Environment Data Structure

- Data structure associating variable keys with Values

```
(define-type-alias Binding (Symbol * Value))
(define-type-alias Env (Listof Binding))

;; Environment is either empty or extended env
(define mt-env : Env
  empty)
(define (extend [bnd : Binding]
                [env : Env])
        : Env
  (cons bnd env))
```

- Textbook uses hashtables
  - The important thing is the interface

Looking up variables

- Find value for a given variable

Looking up variables

- Find value for a given variable
- Find the **first** binding in the environment

Looking up variables

- Find value for a given variable
- Find the **first** binding in the environment
    - This is important for shadowing

Looking up variables

- Find value for a given variable
- Find the **first** binding in the environment
    - This is important for shadowing
- Idea: get all pairs whose variable is what we're looking for

Looking up variables

- Find value for a given variable
- Find the **first** binding in the environment
  - This is important for shadowing
- Idea: get all pairs whose variable is what we're looking for
  - Then take the first

Looking up variables

- Find value for a given variable
- Find the **first** binding in the environment
  - This is important for shadowing
- Idea: get all pairs whose variable is what we're looking for
  - Then take the first

Looking up variables

- Find value for a given variable
- Find the **first** binding in the environment
  - This is important for shadowing
- Idea: get all pairs whose variable is what we're looking for
  - Then take the first

```
(define (lookup [var : Symbol]
                [env : Env])
        : Value
  (let* ([matching (filter
                    (lambda (bnd) (equal? (fst bnd) var))
                    env)])
    (type-case Env matching
        [(empty)
           (error 'lookup
                  (string-append "Undefined variable: "
                                 (to-string var)))]
        [(cons h t)
           (pairSnd h)])))
```

Intepreting Curly-Lambda with Environments

- We can change the implementation

Intepreting Curly-Lambda with Environments

- We can change the implementation
  - *without changing the surface language*

Intepreting Curly-Lambda with Environments

- We can change the implementation
  - *without changing the surface language*
  - or the expression datatype

Intepreting Curly-Lambda with Environments

- We can change the implementation
  - *without changing the surface language*
  - or the expression datatype
- Programs should run the exact same in both interpreters

Intepreting Curly-Lambda with Environments

- We can change the implementation
  - *without changing the surface language*
  - or the expression datatype
- Programs should run the exact same in both interpreters
- Strategy: add an extra context argument for Environment

Intepreting Curly-Lambda with Environments

- We can change the implementation
  - *without changing the surface language*
  - or the expression datatype
- Programs should run the exact same in both interpreters
- Strategy: add an extra context argument for Environment
  - Pass along with every recursive call

Intepreting Curly-Lambda with Environments

- We can change the implementation
  - *without changing the surface language*
  - or the expression datatype
- Programs should run the exact same in both interpreters
- Strategy: add an extra context argument for Environment
  - Pass along with every recursive call
  - Extend environment for recursive calls with newly defined variable

Intepreting Curly-Lambda with Environments

- We can change the implementation
  - *without changing the surface language*
  - or the expression datatype
- Programs should run the exact same in both interpreters
- Strategy: add an extra context argument for Environment
  - Pass along with every recursive call
  - Extend environment for recursive calls with newly defined variable

Intepreting Curly-Lambda with Environments

- We can change the implementation
  - *without changing the surface language*
  - or the expression datatype
- Programs should run the exact same in both interpreters
- Strategy: add an extra context argument for Environment
  - Pass along with every recursive call
  - Extend environment for recursive calls with newly defined variable

```
(define (interp [env : Env]
                [e : Exp] ) : Value
  (type-case Exp e
             ....))
```

Case: Plus etc.

- Exactly like before, except we have to pass the environment in the recursive call

Case: Plus etc.

- Exactly like before, except we have to pass the environment in the recursive call
- Other operations are similar

Case: Plus etc.

- Exactly like before, except we have to pass the environment in the recursive call
- Other operations are similar

Case: Plus etc.

- Exactly like before, except we have to pass the environment in the recursive call
- Other operations are similar

```
;; {+ e1 e2} evaluates e1 and e2, then adds the results together
    [(Plus l r)
     (+ (interp env l) (interp env r))]
```

Case: Variable

- Can't return an error, because we might have added a deferred substitution to the environment

Case: Variable

- Can't return an error, because we might have added a deferred substitution to the environment
- So we look in the environment and see if there's a value *bound* to x

Case: Variable

- Can't return an error, because we might have added a deferred substitution to the environment
- So we look in the environment and see if there's a value *bound* to x
- If there is return it

Case: Variable

- Can't return an error, because we might have added a deferred substitution to the environment
- So we look in the environment and see if there's a value *bound* to x
- If there is return it
    - Otherwise, variable not found error

Case: Variable

- Can't return an error, because we might have added a deferred substitution to the environment
- So we look in the environment and see if there's a value *bound* to x
- If there is return it
    - Otherwise, variable not found error

Case: Variable

- Can't return an error, because we might have added a deferred substitution to the environment
- So we look in the environment and see if there's a value *bound* to x
- If there is return it
  - Otherwise, variable not found error

```
[(Var x)
     (lookup x env)]
```

A (Wrong) First Attempt: Values

- Define `Value` just like in substitution version

A (Wrong) First Attempt: Values

- Define `Value` just like in substitution version
  - Functions consist of their argument and the body to execute

A (Wrong) First Attempt: Values

- Define `Value` just like in substitution version
    - Functions consist of their argument and the body to execute

A (Wrong) First Attempt: Values

- Define `Value` just like in substitution version
    - Functions consist of their argument and the body to execute

```
(define-type Value
  (numV [num : Number])
  (boolV [b : Boolean])
  (lamV [arg : Symbol]
        [body : Exp]))
```

A Wrong First Attempt: Functions Interp

- As a first attempt, try building functions just like in substitution version

A Wrong First Attempt: Functions Interp

- As a first attempt, try building functions just like in substitution version

A Wrong First Attempt: Functions Interp

- As a first attempt, try building functions just like in substitution version

```
(define (interp env expr)
  (type-case Exp interp
     ....
     [(lamE x body)
       (lamV x body)] ))
```

A Wrong First Attempt: Calls Interp

- Just like before

A Wrong First Attempt: Calls Interp

- Just like before
  - Evaluate argument to value

A Wrong First Attempt: Calls Interp

- Just like before
    - Evaluate argument to value
    - Evaluate function, make sure it's actually a function, and get its parameter and body

A Wrong First Attempt: Calls Interp

- Just like before
  - Evaluate argument to value
  - Evaluate function, make sure it's actually a function, and get its parameter and body
- Interpret function body in the environment with the parameter bound to the argument's value

A Wrong First Attempt: Calls Interp

- Just like before
  - Evaluate argument to value
  - Evaluate function, make sure it's actually a function, and get its parameter and body
- Interpret function body in the environment with the parameter bound to the argument's value
  - Whenever we see the variable in the function body, we'll use its value

A Wrong First Attempt: Calls Interp

- Just like before
  - Evaluate argument to value
  - Evaluate function, make sure it’s actually a function, and get its parameter and body
- Interpret function body in the environment with the parameter bound to the argument’s value
  - Whenever we see the variable in the function body, we’ll use its value

A Wrong First Attempt: Calls Interp

- Just like before
  - Evaluate argument to value
  - Evaluate function, make sure it's actually a function, and get its parameter and body
- Interpret function body in the environment with the parameter bound to the argument's value
  - Whenever we see the variable in the function body, we'll use its value

```
(define (interp env expr)
  (type-case Exp interp
  ....
  [(appE fun arg)
     ;; Interpret the function and arg
     (let* ([argVal (interp env arg)]
            [funVal (interp env fun)])
       (type-case Value funVal
         [(lamV var body) ;; Extend current env with argument value
          (interp (extend var argVal env) body)]
         [else
          (error 'interp "Tried to call non-function")]))]
```

The problem

- With a lambda, there could have been many substitutions that were applied to its free variables

The problem

- With a lambda, there could have been many substitutions that were applied to its free variables
  - With substitutions, the variables get replaced in the lambda

The problem

- With a lambda, there could have been many substitutions that were applied to its free variables
  - With substitutions, the variables get replaced in the lambda
  - With environments, the variables aren't replaced *until we interpret the body*

The problem

- With a lambda, there could have been many substitutions that were applied to its free variables
    - With substitutions, the variables get replaced in the lambda
    - With environments, the variables aren't replaced *until we interpret the body*
- When we actually go to interpret the body, we don't have the environment that the function was created in

The problem

- With a lambda, there could have been many substitutions that were applied to its free variables
  - With substitutions, the variables get replaced in the lambda
  - With environments, the variables aren't replaced *until we interpret the body*
- When we actually go to interpret the body, we don't have the environment that the function was created in
  - Just the environment from the time of the call

The problem

- With a lambda, there could have been many substitutions that were applied to its free variables
  - With substitutions, the variables get replaced in the lambda
  - With environments, the variables aren't replaced *until we interpret the body*
- When we actually go to interpret the body, we don't have the environment that the function was created in
  - Just the environment from the time of the call
- We've implemented **dynamic scoping** by accident!

Terminology

- *Static scope* means a function never changes once it has been **defined**

Terminology

- *Static scope* means a function never changes once it has been **defined**
    - We can still create functions dynamically

Terminology

- *Static scope* means a function never changes once it has been **defined**
  - We can still create functions dynamically
  - But once created they’re static

Terminology

- *Static scope* means a function never changes once it has been **defined**
    - We can still create functions dynamically
    - But once created they're static
        - e.g. any undefined variables get their values from the environment *where the function was defined*

Terminology

- *Static scope* means a function never changes once it has been **defined**
    - We can still create functions dynamically
    - But once created they're static
        - e.g. any undefined variables get their values from the environment *where the function was defined*
- *Dynamic scope* means that any free (undefined) variables in a function get their values from the environment when the function was *called* (not defined)

Implementing Static Scope: Closures

- A function should be *closed* over its environment at the point it's created (interpreted)

Implementing Static Scope: Closures

- A function should be *closed* over its environment at the point it's created (interpreted)
- So we add an extra piece of data to the `Value` variant for functions: the environment at the time of creation

Implementing Static Scope: Closures

- A function should be *closed* over its environment at the point it's created (interpreted)
- So we add an extra piece of data to the `Value` variant for functions: the environment at the time of creation
  - The combination of a function variable+body and an environment is called a **closure**

Implementing Static Scope: Closures

- A function should be *closed* over its environment at the point it's created (interpreted)
- So we add an extra piece of data to the `Value` variant for functions: the environment at the time of creation
  - The combination of a function variable+body and an environment is called a **closure**
- Closures give environment interpreters the same behavior as substitution interpreters

The New Value Type

- Value version of functions contains an environment in addition to its variable and body

The New Value Type

- Value version of functions contains an environment in addition to its variable and body

The New Value Type

- Value version of functions contains an environment in addition to its variable and body

```
(define-type Value
  (numV [num : Number])
  (boolV [b : Boolean])
  ;; Like lamV but with an environment
  (closureV [arg : Symbol]
            [body : Exp]
            [env : Env]))
```

Properly Interpreting Functions

- When we interpret a lambda, *package the current environment up with it*

Properly Interpreting Functions

- When we interpret a lambda, *package the current environment up with it*
  - This is what lets us dynamically create new functions

Properly Interpreting Functions

- When we interpret a lambda, *package the current environment up with it*
    - This is what lets us dynamically create new functions
    - For top-level functions, this is the empty environment

Properly Interpreting Functions

- When we interpret a lambda, *package the current environment up with it*
  - This is what lets us dynamically create new functions
  - For top-level functions, this is the empty environment

Properly Interpreting Functions

- When we interpret a lambda, *package the current environment up with it*
    - This is what lets us dynamically create new functions
    - For top-level functions, this is the empty environment

```
(define (interp env expr)
  (type-case Exp interp
     ....
     [(lamE x body)
       (closureV x body env) ;;<------
       ] ))
```

Properly Interpreting Calls

- When we call a function, extend *the environment that was packaged up with it*

Properly Interpreting Calls

- When we call a function, extend *the environment that was packaged up with it*
    - Any free variables in the body get values from the place the closure was constructed

Properly Interpreting Calls

- When we call a function, extend *the environment that was packaged up with it*
    - Any free variables in the body get values from the place the closure was constructed

Properly Interpreting Calls

- When we call a function, extend *the environment that was packaged up with it*
  - Any free variables in the body get values from the place the closure was constructed

```
(define (interp env expr)
  (type-case Exp interp
  ....
  [(appE fun arg)
     ;; Argument and function are interpreted in the same environment
     ;; e.g. NOT the environment from the closure
     (let* ([argVal (interp env arg)]
            [funVal (interp env fun)])
       (type-case Value funVal
         [(closureV var body funEnv)
          (let* ([envForCall (extend var argVal funEnv)])
          (interp envForCall body))]
         [else
          (error 'interp "Tried to call non-function")]))]
```

Summary: Dynamic vs. Static Scope

- Free variables in a function body are variables that are not defined/bound in that function body

Summary: Dynamic vs. Static Scope

- Free variables in a function body are variables that are not defined/bound in that function body
- Static scope gives free variables values from the environment when the function was *constructed*

Summary: Dynamic vs. Static Scope

- Free variables in a function body are variables that are not defined/bound in that function body
- Static scope gives free variables values from the environment when the function was *constructed*
- Dynamic scope gives variables values from the environment when the function was *called*

Example: Static Scope

Example: Static Scope

```
{let1 double {lam x {* x 2}}
  {let1 quadruple {lam y {double {double y}}}
    {let1 double 2
      {quadruple 3}}}}
```

- `{lam y {double {double x}}}` has one free variable, `double`

Example: Static Scope

```
{let1 double {lam x {* x 2}}
  {let1 quadruple {lam y {double {double y}}}
    {let1 double 2
      {quadruple 3}}}}
```

- `{lam y {double {double x}}}` has one free variable, `double`
  - Functions and variables are in *the same namespace* in Curly-Lambda

Example: Static Scope

```
{let1 double {lam x {* x 2}}
  {let1 quadruple {lam y {double {double y}}}
    {let1 double 2
      {quadruple 3}}}}
```

- `{lam y {double {double x}}}` has one free variable, `double`
  - Functions and variables are in *the same namespace* in Curly-Lambda
- `{lam y {double {double x}}}` evaluates to closure

Example: Static Scope

```
{let1 double {lam x {* x 2}}
  {let1 quadruple {lam y {double {double y}}}
    {let1 double 2
      {quadruple 3}}}}
```

- `{lam y {double {double x}}}` has one free variable, `double`
  - Functions and variables are in *the same namespace* in Curly-Lambda
- `{lam y {double {double x}}}` evaluates to closure
  - Body is `{double {double x}}`

Example: Static Scope

```
{let1 double {lam x {* x 2}}
  {let1 quadruple {lam y {double {double y}}}
    {let1 double 2
      {quadruple 3}}}}
```

- `{lam y {double {double x}}}` has one free variable, `double`
  - Functions and variables are in *the same namespace* in Curly-Lambda
- `{lam y {double {double x}}}` evaluates to closure
  - Body is `{double {double x}}`
  - Env is `double := {lam x {* x 2}}`

Example: Static Scope

```
{let1 double {lam x {* x 2}}
  {let1 quadruple {lam y {double {double y}}}
    {let1 double 2
      {quadruple 3}}}}
```

- `{lam y {double {double x}}}` has one free variable, `double`
  - Functions and variables are in *the same namespace* in Curly-Lambda
- `{lam y {double {double x}}}` evaluates to closure
  - Body is `{double {double x}}`
  - Env is `double := {lam x {* x 2}}`
- Env at call:

Example: Static Scope

```
{let1 double {lam x {* x 2}}
  {let1 quadruple {lam y {double {double y}}}
    {let1 double 2
      {quadruple 3}}}}
```

- `{lam y {double {double x}}}` has one free variable, `double`
  - Functions and variables are in *the same namespace* in Curly-Lambda
- `{lam y {double {double x}}}` evaluates to closure
  - Body is `{double {double x}}`
  - Env is `double := {lam x {* x 2}}`
- Env at call:
  - `double := 2`

Example: Static Scope

```
{let1 double {lam x {* x 2}}
  {let1 quadruple {lam y {double {double y}}}
    {let1 double 2
      {quadruple 3}}}}
```

- `{lam y {double {double x}}}` has one free variable, `double`
  - Functions and variables are in *the same namespace* in Curly-Lambda
- `{lam y {double {double x}}}` evaluates to closure
  - Body is `{double {double x}}`
  - Env is `double := {lam x {* x 2}}`
- Env at call:
  - `double := 2`
  - `quadruple := {lam y {double {double x}}}`

Example: Static Scope

```
{let1 double {lam x {* x 2}}
  {let1 quadruple {lam y {double {double y}}}
    {let1 double 2
      {quadruple 3}}}}
```

- `{lam y {double {double x}}}` has one free variable, `double`
  - Functions and variables are in *the same namespace* in Curly-Lambda
- `{lam y {double {double x}}}` evaluates to closure
  - Body is `{double {double x}}`
  - Env is `double := {lam x {* x 2}}`
- Env at call:
  - `double := 2`
  - `quadruple := {lam y {double {double x}}}`
  - `double := {lam x {* x 2}}`

Example: Static Scope ctd

Example: Static Scope ctd

```
{let1 double {lam x {* x 2}}
  {let1 quadruple {lam y {double {double x}}}
    {let1 double 2
      {quadruple 3}}}}
```

- Call evaluates `quadruple` to closure

Example: Static Scope ctd

```
{let1 double {lam x {* x 2}}
  {let1 quadruple {lam y {double {double x}}}
    {let1 double 2
      {quadruple 3}}}}
```

- Call evaluates `quadruple` to closure
- Finally evaluates `{double {double x}}`

Example: Static Scope ctd

```
{let1 double {lam x {* x 2}}
  {let1 quadruple {lam y {double {double x}}}
    {let1 double 2
      {quadruple 3}}}}
```

- Call evaluates `quadruple` to closure
- Finally evaluates `{double {double x}}`
    - In extended closure environment:

Example: Static Scope ctd

```
{let1 double {lam x {* x 2}}
  {let1 quadruple {lam y {double {double x}}}
    {let1 double 2
      {quadruple 3}}}}
```

- Call evaluates `quadruple` to closure
- Finally evaluates `{double {double x}}`
    - In extended closure environment:
        - `x := 3`

Example: Static Scope ctd

```
{let1 double {lam x {* x 2}}
  {let1 quadruple {lam y {double {double x}}}
    {let1 double 2
      {quadruple 3}}}}
```

- Call evaluates `quadruple` to closure
- Finally evaluates `{double {double x}}`
    - In extended closure environment:
        - `x := 3`
        - `double := {lam x {* x 2}}`

Example: Static Scope ctd

```
{let1 double {lam x {* x 2}}
  {let1 quadruple {lam y {double {double x}}}
    {let1 double 2
      {quadruple 3}}}}
```

- Call evaluates `quadruple` to closure
- Finally evaluates `{double {double x}}`
    - In extended closure environment:
        - `x := 3`
        - `double := {lam x {* x 2}}`
- Result is 12

Example: Dynamic Scope

Example: Dynamic Scope

```
{let1 double {lam x {* x 2}}
  {let1 quadruple {lam y {double {double x}}}
    {let1 double 2
      {quadruple 3}}}}
```

- `{lam y {double {double x}}}` evaluates to closure

Example: Dynamic Scope

```
{let1 double {lam x {* x 2}}
  {let1 quadruple {lam y {double {double x}}}
    {let1 double 2
      {quadruple 3}}}}
```

- `{lam y {double {double x}}}` evaluates to closure
  - Doesn’t save environment

Example: Dynamic Scope

```
{let1 double {lam x {* x 2}}
  {let1 quadruple {lam y {double {double x}}}
    {let1 double 2
      {quadruple 3}}}}
```

- `{lam y {double {double x}}}` evaluates to closure
  - Doesn't save environment
- Call evaluates `{double {double x}}`

Example: Dynamic Scope

```
{let1 double {lam x {* x 2}}
  {let1 quadruple {lam y {double {double x}}}
    {let1 double 2
      {quadruple 3}}}}
```

- `{lam y {double {double x}}}` evaluates to closure
  - Doesn’t save environment
- Call evaluates `{double {double x}}`
  - In extended *call site* environment:

Example: Dynamic Scope

```
{let1 double {lam x {* x 2}}
  {let1 quadruple {lam y {double {double x}}}
    {let1 double 2
      {quadruple 3}}}}
```

- `{lam y {double {double x}}}` evaluates to closure
  - Doesn't save environment
- Call evaluates `{double {double x}}`
  - In extended *call site* environment:
    - `x := 3`

Example: Dynamic Scope

```
{let1 double {lam x {* x 2}}
  {let1 quadruple {lam y {double {double x}}}
    {let1 double 2
      {quadruple 3}}}}
```

- `{lam y {double {double x}}}` evaluates to closure
  - Doesn't save environment
- Call evaluates `{double {double x}}`
  - In extended *call site* environment:
    - `x := 3`
    - `double := 2`

Example: Dynamic Scope

```
{let1 double {lam x {* x 2}}
  {let1 quadruple {lam y {double {double x}}}
    {let1 double 2
      {quadruple 3}}}}
```

- `{lam y {double {double x}}}` evaluates to closure
  - Doesn't save environment
- Call evaluates `{double {double x}}`
  - In extended *call site* environment:
    - `x := 3`
    - `double := 2`
    - `quadruple := {lam y {double {double x}}}`

Example: Dynamic Scope

```
{let1 double {lam x {* x 2}}
  {let1 quadruple {lam y {double {double x}}}
    {let1 double 2
      {quadruple 3}}}}
```

- `{lam y {double {double x}}}` evaluates to closure
  - Doesn't save environment
- Call evaluates `{double {double x}}`
  - In extended *call site* environment:
    - `x := 3`
    - `double := 2`
    - `quadruple := {lam y {double {double x}}}`
    - `double := {lam x {* x 2}}`

Example: Dynamic Scope

```
{let1 double {lam x {* x 2}}
  {let1 quadruple {lam y {double {double x}}}
    {let1 double 2
      {quadruple 3}}}}
```

- `{lam y {double {double x}}}` evaluates to closure
  - Doesn't save environment
- Call evaluates `{double {double x}}`
  - In extended *call site* environment:
    - `x := 3`
    - `double := 2`
    - `quadruple := {lam y {double {double x}}}`
    - `double := {lam x {* x 2}}`
- Dynamic type error

Example: Dynamic Scope

```
{let1 double {lam x {* x 2}}
  {let1 quadruple {lam y {double {double x}}}
    {let1 double 2
      {quadruple 3}}}}
```

- `{lam y {double {double x}}}` evaluates to closure
  - Doesn't save environment
- Call evaluates `{double {double x}}`
  - In extended *call site* environment:
    - `x := 3`
    - `double := 2`
    - `quadruple := {lam y {double {double x}}}`
    - `double := {lam x {* x 2}}`
- Dynamic type error
  - Can't call 2 as a function

But Professor, When Will I Ever Use This?

Static Scoping in the Wild

Python:

Static Scoping in the Wild

Python:

Static Scoping in the Wild

Python:

```
timesTwo = lambda x : 2 * x
quadruple = lambda y : timesTwo(timesTwo(y))
def mainFun(x):
    timesTwo = 2.0
    return quadruple(x)
return mainFun(3)
```

12

Result:

12

Static Scoping in the Wild

JavaScript:

Static Scoping in the Wild

JavaScript:

Static Scoping in the Wild

JavaScript:

```
var timesTwo = function (x) { return x * 2 };
var quadruple =
    function (x) {return timesTwo(timesTwo(x)) };
function mainFun(x){
    var timesTwo = 2.0;
    return quadruple(x)}
return mainFun(3)
```

12

Result:

12

Async in JavaScript

- The entire JS approach depends on static scope

Async in JavaScript

- The entire JS approach depends on static scope
- From the w3schools async tutorial

Async in JavaScript

- The entire JS approach depends on static scope
- From the w3schools async tutorial

Async in JavaScript

- The entire JS approach depends on static scope
- From the w3schools async tutorial

```
async function myFunction() {
  return "Hello";
}
myFunction().then(
  function(value) {myDisplayer(value);}
);
```

- `myFunction.then` is a higher order function

Async in JavaScript

- The entire JS approach depends on static scope
- From the w3schools async tutorial

```
async function myFunction() {
  return "Hello";
}
myFunction().then(
  function(value) {myDisplayer(value);}
);
```

- `myFunction.then` is a higher order function
  - Takes in another function as an argument

Async in JavaScript

- The entire JS approach depends on static scope
- From the w3schools async tutorial

```
async function myFunction() {
  return "Hello";
}
myFunction().then(
  function(value) {myDisplayer(value);}
);
```

- `myFunction.then` is a higher order function
    - Takes in another function as an argument
- They call the function argument the *callback*

Async in JavaScript

- The entire JS approach depends on static scope
- From the w3schools async tutorial

```
async function myFunction() {
  return "Hello";
}
myFunction().then(
  function(value) {myDisplayer(value);}
);
```

- `myFunction.then` is a higher order function
    - Takes in another function as an argument
- They call the function argument the *callback*
- `function(value)` is just the Javascript syntax for lambda

Async in JavaScript

- The entire JS approach depends on static scope
- From the w3schools async tutorial

```
async function myFunction() {
  return "Hello";
}
myFunction().then(
  function(value) {myDisplayer(value);}
);
```

- `myFunction.then` is a higher order function
  - Takes in another function as an argument
- They call the function argument the *callback*
- `function(value)` is just the Javascript syntax for lambda
  - Dynamically creates the function that is run when `myFunction` actually runs

Async in JavaScript

- The entire JS approach depends on static scope
- From the w3schools async tutorial

```
async function myFunction() {
  return "Hello";
}
myFunction().then(
  function(value) {myDisplayer(value);}
);
```

- `myFunction.then` is a higher order function
    - Takes in another function as an argument
- They call the function argument the *callback*
- `function(value)` is just the Javascript syntax for lambda
    - Dynamically creates the function that is run when `myFunction` actually runs
- Concurrency in JS is mostly just syntactic sugar for lambda/higher-order functions